Subject: Module 5 Assignment - Order Tracking of Mobius Propeller Data

Date: May 1, 2025 (Submitted)

Order Tracking of Mobius Propeller Data

The Matlab code for this problem is listed in Appendix 1.

Sound Pressure Level Versus Microphone Placement Angle

Figure 1 shows the sound pressure generated by the Mobius propeller versus the angle of microphone placement.

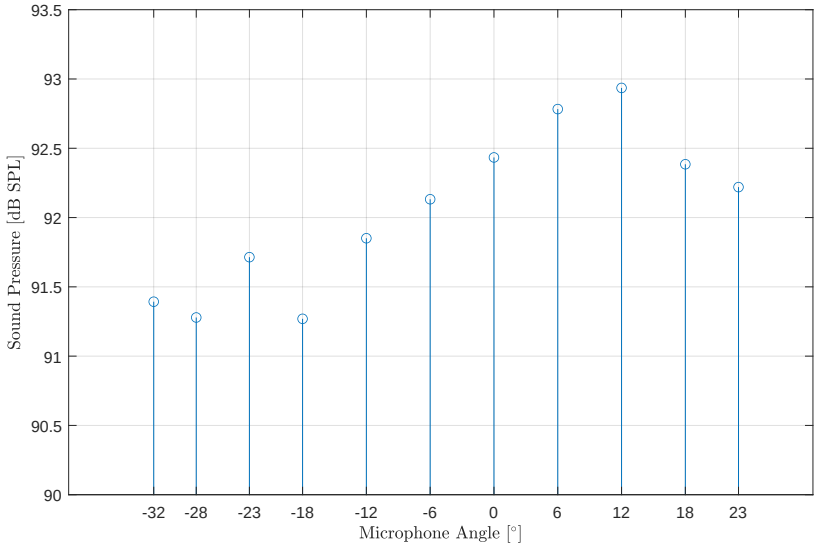

Figure 1: Sound pressure level versus microphone placement angle.

The highest sound pressure levels occur around the 0° plane, when the propeller is in the same plane as the microphone. The highest sound pressure, approximately 93 dB SPL, occurs at 12°, is radiated above the plane of the propeller and the drone itself.

Trigger Signal and Representative Pressure Recording

Figure 2 shows the angular position trigger signal and the pressure recording for a measurement angle of 0°..

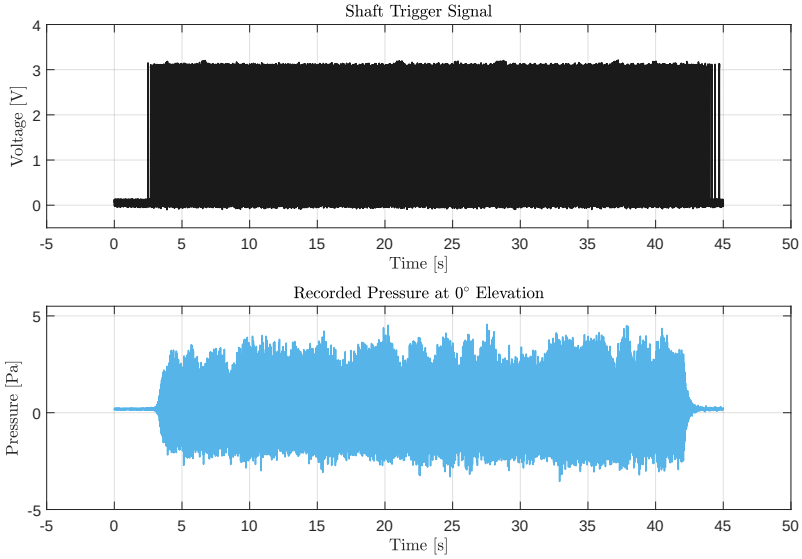

Figure 2: Trigger signal and a pressure recording at 0°.

ph

1 Appendix - Matlab Code for Problem 1

```
1
2
3
4  %% Synopsis
5
6  % Problem 5 - Order Tracking of Mobius Propeller Data
7
8  % See Lecture 26
9  %   ""D:\15 Downloads\00 GitHub\ACS_547\35 Lectures\26 Monday, April 21, 2025\Lecture 26 - Shafting and
       bearings.pptx""
10
11
12
13 %% Note(s)
14
15 % Angle position is encoded here using a once-per-revolution pulse.
16
17
18
19 %% To Do
20
21 % Include labels on plots for each harmonic order (of blade rotation rate).
22
23
24
25 %% Environment
26
27 close all; clear; clc;
28 % restoredefaultpath;
29
30 addpath( genpath( './00 Support' ), '-begin' );
31
32 % set( groot, 'DefaultFigurePosition', [ 230 730  750  500 ] );  % x, y, width, height
33
34 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
35 set( 0, 'DefaultFigureWindowStyle', 'normal' );
36 set( 0, 'DefaultLineLineWidth', 1.0 );
37 set( 0, 'DefaultTextInterpreter', 'Latex' );
38
39 format ShortG;
40
41 color_map = slanCM( 'ColorBlind', 4 );
42
43 pause( 1 );
44
45
46
47 %% Load Data
48
49 load( 'mobius_prop_data.mat' );  % Variable(s):  fs;  mic_angles_degrees;  p;  trigger
50
51 % fs - Sample rate of 80 kHz (scalar).
52 % mic_angles_degrees - Microphone angles relative to horizontal plane (11-by-1;  23, 18, 6, 0, -6, -12,
       -18, -23, -28, and -32).
53 % p - Recorded pressures for the corresponding microphone angles (3,600,000-by-11).
54 % trigger - Shaft rotation synchronization signal (3,600,000-by-11).
55
56
57
58 %% RMS dB SPL Per Pressure Recording
59
60 % p_rms = 20*log10( rms( p ) / 20e-6 );
61 %
62 % figure( 'Name', 'RMS dB SPL Per Pressure Recording' ); ...
63 %     stem( mic_angles_degrees, fliplr( p_rms ) );  grid on;
64 %         xlabel( 'Microphone Angle [$^\circ$]' );
65 %         xticks( fliplr( mic_angles_degrees ) );
66 %         ylabel( 'Sound Pressure [dB SPL]' );
67
68
69
70 %% Pressure Recordings
71
72 % figure( 'Name', 'Pressure Recordings' ); ...
73 %     plot( p( :, 1 ) );  grid on;
74 %         xlabel( 'Time [WU]' );  ylabel( 'Sound Pressure [Pa]' );
75
76
77
78 %% Visualize Experimental Data
79
80 net_time = size( p, 1 ) / fs;  % 45 seconds
81
82 time_indices = ( 0:1:( numel( trigger ) - 1 ) ) ./ fs;
83     time_indices = time_indices(:);
84
85
86 figure( 'Name', 'Trigger signal and Data for Horizontal Plane' ); ...
87
```

```
    h1 = subplot( 2, 1, 1 ); ...
        plot( time_indices, trigger, 'Color', color_map( 1, : ) );  grid on;
            xlabel( 'Time [s]' );  ylabel( 'Voltage [V]' );  title( 'Shaft Trigger Signal' );
            ylim( [ -0.5 4 ] );
    h2 = subplot( 2, 1, 2 ); ...
        plot( time_indices, p( :, 5 ), 'Color', color_map( 2, : ) );  grid on;
            xlabel( 'Time [s]' );  ylabel( 'Pressure [Pa]' );  title( 'Recorded Pressure at 0$^\circ$
                Elevation' );
            ylim( [ -5 5.5 ] );

    linkaxes( [ h1 h2 ], 'x' );
        xlim( [ -5 50 ] );

return

%% Spectrogram

% signal = p(:, 1);
%
% frame_length = 8192;
%     frame_hop = floor( 0.20 * frame_length );
%
% a_spectrogram = spectrogram_September_26_2023( signal, frame_length, frame_hop, fs, 0 );
%
% figure( 'Name', 'Spectrogram of Pressure Signal' ); ...
%
%     imagesc( a_spectrogram.time_indices, a_spectrogram.Sxx.frequencies, 20*log10( sqrt(a_spectrogram.
    Sxx.spectrum)./20e-6 ), [ 0 85 ] );
%         labelColorbar( 'Power Spectral Density [dB re: $\frac{20 \mu Pa^2}{Hz}$]' );  grid on;
%         colormap parula;  % Option:  Turbo
%         set( gca, 'GridColor', 'w', 'GridAlpha', 0.4 );
%     xlabel( 'Time [s]' );  ylabel( 'Frequency [Hz]' );  title( 'Order Tracking' );
%     set( gca, 'ydir', 'normal' );
%     ylim( [ 0 1e3] );



%% Locate the Leading Edges of the Trigger Data

% Smooth data to reduce noise;  supports finding the leading eges.
smoothed_trigger_data = [ 0;  diff( smooth( trigger, 'moving', 5 ) ) ];

% Threshold smoothed data to located leading edges.
threshold = 0.5;
    edge_indices = find( threshold  <  smoothed_trigger_data );
        leading_edges = edge_indices( 1:2:end );

temp = diff( leading_edges );
    [ indices_to_remove, indices ] = find( temp == 2 );
        clear temp;

temp = leading_edges;
    temp( indices_to_remove ) = [ ];

leading_edges = temp;
    clear temp;

time_indices_leading_edge = leading_edges ./ fs;


figure( ); ...

    plot( trigger( 1:1:end ), 'Color', 'k' );  hold on;
    %
    for index = 1:1:numel( leading_edges )
        line( [ leading_edges( index ) leading_edges( index ) ], [ 0 4 ], 'Color', 'b' );  grid on;
    end

    axis( [ 1 numel( trigger )  -0.5  4.5 ] );



%% Segment Microphone Recording Channels

% Channel 1:  0 degrees
% Channel 2:  18 degrees
% Channel 3:  12 degrees
% Channel 4:  6 degrees
% Channel 5:  0 degrees
% Channel 6:  -6 degrees
% Channel 7:  -12 degrees
% Channel 8:  -18 degrees
% Channel 9:  -23 degrees
% Channel 10:  -28 degrees
% Channal 11:  -32 degrees


start_indices = 1:1:( numel( leading_edges ) - 1 );  end_indices = 2:1:numel( leading_edges );

frame_set_indices = [ leading_edges( start_indices(:) )  leading_edges( end_indices(:) ) ];
    number_of_revolutions = size( frame_set_indices, 1 );

```

```
178
179 for channel_index = 1:1:size( p, 2 )
180
181     for frame_index = 1:1:size( frame_set_indices, 1 )
182
183         channel_segments{ frame_index, channel_index } = p( frame_set_indices( frame_index, 1 ):1:
                frame_set_indices( frame_index, 2 ), channel_index );
184
185         slope_theta = (2*pi) ./ size( channel_segments{ frame_index, channel_index }, 1 );
186             theta{ frame_index, channel_index } = slope_theta .* ( 0:1:size( channel_segments{
                    frame_index, channel_index }, 1 ) - 1 );
187
188     end
189
190 end
191
192
193
194 %% Compute Instantaneous RPM
195
196 for channel_index = 1:1:size( p, 2 )
197     for frame_index = 1:1:size( frame_set_indices, 1 )
198         rpm_estimate{ frame_index, channel_index } = 60 * ( 1 / ( numel( channel_segments{ frame_index,
                channel_index } ) ./ fs ) );
199     end
200 end
201
202
203 figure( ); ...
204
205     plot( [ rpm_estimate{ :, 1 } ], 'Color', 'k' );  grid on;
206     xlabel( 'Revolution [WU]' );  ylabel( 'RPM' );
207     axis( [ 1 number_of_revolutions  0 3.75e3 ] );
208     shg;
209
210
211
212 %% Compute Fourier Coefficients
213
214 An = nan( size( frame_set_indices, 1 ), 11 );  Bn = An;
215
216
217 TRACKING_ORDER = 1;
218
219 for channel_index = 1:1:size( p, 2 )
220
221     for frame_index = 1:1:size( frame_set_indices, 1 )
222
223         M = numel( channel_segments{ frame_index, channel_index } );
224             m = 0:1:( M - 1 );
225
226         An( frame_index, channel_index ) = ...
227             2/M * channel_segments{ frame_index, channel_index }.' * cos( TRACKING_ORDER .* theta{
                    frame_index, channel_index } ).';
228
229         Bn( frame_index, channel_index ) = ...
230             2/M * channel_segments{ frame_index, channel_index }.' * sin( TRACKING_ORDER .* theta{
                    frame_index, channel_index } ).';
231
232     end
233
234 end
235
236
237 figure( ); ...
238
239     h1 = subplot( 3, 1, 1 ); ...
240         plot( nan, nan );  hold on;
241         %
242         for channel_index = 1:1:size( p, 2 )
243             plot( An( :, channel_index ), 'Color', 'b' );
244         end
245         %
246         xlabel( 'Revolution [WU]' );  ylabel( 'An' );
247         %
248         grid on;  axis( [ 1 number_of_revolutions  -1 1 ] );
249
250     h2 = subplot( 3, 1, 2 ); ...
251         plot( nan, nan );  hold on;
252         %
253         for channel_index = 1:1:size( p, 2 )
254             plot( Bn( :, channel_index ), 'Color', 'r' );  grid on;
255         end
256         %
257         xlabel( 'Revolution [WU]' );  ylabel( 'Bn' );
258         %
259         grid on;  axis( [ 1 number_of_revolutions  -1 1 ] );
260
261     h3 = subplot( 3, 1, 3 ); ...
262         plot( nan, nan );  hold on;
263         %
264         for channel_index = 1:1:size( p, 2 )
```

```
            plot( 20*log10( sqrt( An( :, channel_index ).^2 + Bn( :, channel_index ).^2 ) ./ 20e-6 ), 'Color', 'k' );
        end
        %
        xlabel( 'Revolution [WU]' );  ylabel( 'Sound Pressure dB re:20e-6 Pa' );
        %
        grid on;  axis( [ 1 number_of_revolutions  50 110 ] );

    shg;



%% Clean-up

% if ( ~isempty( findobj( 'Type', 'figure' ) ) )
%     monitors = get( 0, 'MonitorPositions' );
%         if ( size( monitors, 1 ) == 1 )
%             autoArrangeFigures( 3, 4, 1 );
%         elseif ( 1 < size( monitors, 1 ) )
%             autoArrangeFigures( 2, 2, 2 );
%         end
% end

fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );



%% Reference(s)

% https://www.mathworks.com/help/signal/ref/ordertrack.html#bvaw5ra-1-x

% https://www.mathworks.com/matlabcentral/answers/1439884-evaluate-rising-edge-sample-of-a-signal
```